

CUDA

Instructions primarily hosted by CPUs, cuda help compiler to take part of the job and offload it to GPU

Cuda $\rightarrow$ CPU launch Threads

GPU DRAM $\rightarrow$ CPU Memory to read

CUDA = Integrated host + C program
Serial & Modestly Parallel by C code
High parallel by kernel

kernel $\llcorner \llcorner nBlk, nTid \gg \gg$ (args); $\Leftrightarrow$ Serial Code

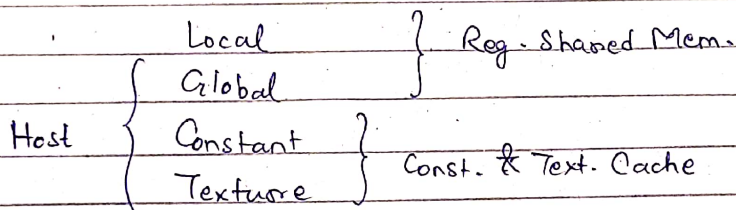
kernel return void

Cuda manages copy to and from cpu memory and gpu memory

Cuda launches threads in blocks and grids (come asch and matrices)

Thread ID $\rightarrow$ Data worked on / Data loc

Grid = 3D Blocks ; Block = 3D Threads



Heterogenous Cuda Program

Unified Memory (CPU + GPU $\leftrightarrow$ Memory)

Memory Architecture

Memory	Loc on/off Chip	Cached	Access	Scope	Lifetime
Registers	On	no	R/W	1 Th	Thread
Local	Off	Yes	R/W	1 Th	Thread
Shared	On	no	R/W	all th in block	Block
Global	Off	Yes	R/W	all th + host	Host alloc
Const.	Off	Yes	R	all th + host	Host alloc
Texture	Off	Yes	R	all th + host	Host alloc

Refs: nvidia cuda-c-programming-guide
nvidia volta-architecture

Cuda Prog

Heterogeneous = CPU + GPU system
 ↓ ↓
 serial part massively parallel part

CUDA Toolkit = platform + prog model

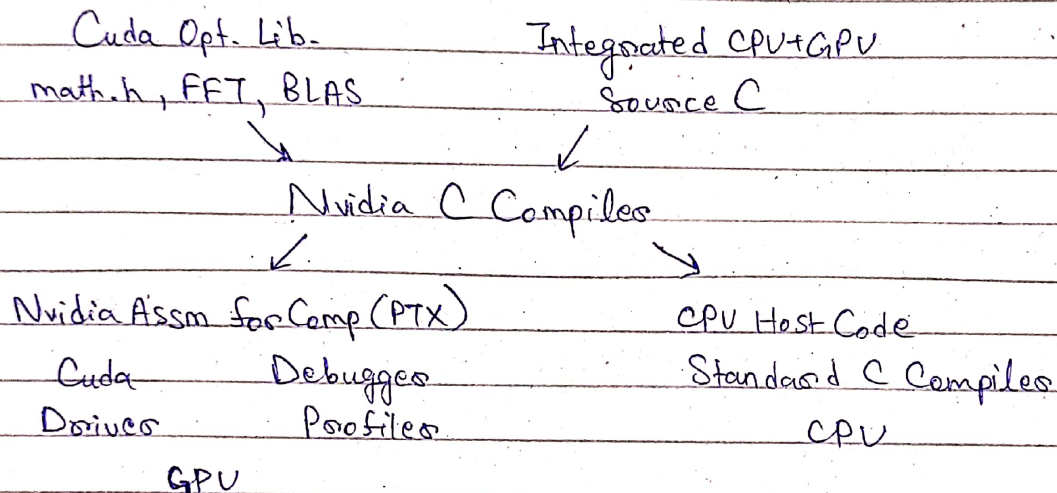
Install CUDA SDK and Debugger

#: nvcc VectorAdd.cu -o VectorAdd

#: ./VectorAdd

No. of threads are specified by kernel spec. ($nBlk \times nTid$)

Thread exe in cores is scheduled by SM level scheduler



```
cudaMalloc (&ptr, N * sizeof (float));
```

```
cudaMemcpy (device, host, size, cudaMemcpyHostToDevice);
```

```
vecAdd <<< gridSize, blockSize >>> (d-A, d-B, d-C, N);
```

```
--global-- void vecAdd (float *a, *b, *c, int n) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < n) c[id] = a[id] + b[id];
}
```

Kernel

--global-- : execution space specifier

<< gridSize, blockSize >> : execution config syntax

Global thread id for each kernel

$id = blockDim.x * blockDim.y * blockDim.z + threadIdx.x$; // unique for each thread

Global:

- Executed on device (gpu)
- Callable from host (cpu)
- Device to Device (cuda dyn parallelism) (comp. capa. ≥ 3.2)

A call to global func. as async; returns before execution finish

--device-- : called and exe from device only

--host-- : called and exe from host only (default)

global & host cannot be used together

device & host can be used together, func compiled for both

Func << Dg, Db, Ns, S >> (param);

dim3 for
block size
 $Dg.x * Dg.y * Dg.z$
= no. of blocks

dim3 as
 $Db.x * Db.y * Db.z$
= no. of th / block

size_t
no. of bytes in shared memory
dynamically alloc / block
def = 0

S = associated stream of instructions

dim3 gridSize(8, 8, 2); // unspecified dim is init to 1

Ⓢ Block and Thread Relations

cudaMalloc() and cudaFree() // managed by cpu, operated by gpu

void ** devPtrs, size_t count

void * devPtrs

cudaMemcpy(void * dst, void * src, size_t nbytes, enum cudaMemcpyKind);
direction

→ blocks CPU thread (until finish)

H2D, D2H, D2D



Scheduling and Memory

C Prog. Seg. Exe:

Serial Code $\rightarrow$ Host

Host launches `kernel()` $\rightarrow$ Device (grid * blocks)

Device launch async $\rightarrow$ Host

Host $\rightarrow$...

Max no. of blocks in x-dim in grid is = `INT_MAX`

Each block max threads = 1024

② Scalability and Scheduling of Threads w.r.t. Cores is imp.

Cuda thread $\rightarrow$ Cuda core

Cuda th. block $\rightarrow$ Cuda SM // Streaming Multi-processors

Cuda kernel grid $\rightarrow$ Cuda-enabled GPU

No. of th. in block $>$ No. of cores in SM

Schedule threads in SM ; Mem Req. of block and SM

Warp : Group of threads scheduled together on a SM ; fixed by gpu arch

warp-by-warp execution on SM cores

Limited to 32 in modern gpu (hw limit)

Restricted to particular block

SM : creates, manages, schedules and executes th. at warp-lvl granularity
each warp of contiguous thread ids

all th. in warp exe same inst.

vector-processor level arch works

if threads of warp diverge in exe path, each branch exe serially

warp coalesces global memory accesses

SM does context switch of warp (like fetch memory).

Data stored in reg. file for easier resume.

Ideally, $\text{blockSize} = n * \text{warp-size}$
avoid thread divergence

Thread Execution

Avoid thread divergence (using if-else) since they'll be exe serially adding latency, idle cores.

if ($i \% 2 == 0$) $x[i] = 5$; $\rightarrow x[i] = (1 - i \% 2) * 5 + (i \% 2) * 10$;
else $x[i] = 10$;

-- sync thread (); sync on barrier thread // block-level
not allow^{til} other threads have reached this point of exe
Deadlock

Diverge at warp-level, block-level (diff. inst.)

Launch diff. kernels

cudaMemcpy, cudaMalloc do implicit sync

Block Scheduling

Per-thread $\leftrightarrow$ Reg. OR local memory (onchip, uncached)
Per-block $\leftrightarrow$ Shared Memory (onchip, small, fast) // ~~L1~~ L1
Per-device : kernel $\leftrightarrow$ Global Memory
offchip, large, uncached, kernel I/O,
persistent across kernel launches

Persisting Access vs Streaming Access ; Cuda kern $\rightarrow$ Global data

SM $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ GPU Mem $\rightarrow$ System Mem

CGMA ratio

Global Memory Access by Warp

Const. Mem = 64KB, cached, few distinct locs

Texture Mem = cached

Local Mem = local to thread, hold auto vars, large storage,
as expensive as global mem.

Shared Mem = 64-96KB, th-sync.

Bank Conflict ; Atomic Func ;

Matrix Multiplication

```
myKernel <<< gridSize, blockSize >>> (...)
```

```
--global-- void matAdd (float A[N][N], ... ) { // B, C
    int i = threadIdx.x, j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}
```

```
int main () {
    int numBlocks = 1;
    dim3 threadsPerBlock (N, N); // N * N * 1
    matAdd <<< numBlocks, threadsPerBlock >>> (A, B, C);
}
```

Matrix Vector Mul : $h[i] = h[i] + (d[h * i + j] + v[j]);$
// coalesce 2D $\rightarrow$ 1D

Shared Memory and Tiling

Help us in cases of large memory access and cache unfriendliness

Check: [nvidia/cuda/cuda-c-programming-guide](#)

[cuda-c-best-practices-guide](#)

OpenACC

- Scientific Computing, Accelerated Programming
- Directive Based Parallelism

GPU have high scalability at cost of thread mgmt. and data handling.
Legacy CPU code $\rightarrow$ GPU code (cuda / opencl)

OpenACC : directive based ~~acc~~ (mention modules) and parallel
(auto parallel compilation)

Multicore CPU and GPU arch ; heterogeneous systems

```
#pragma omp parallel shared (x,y) private (ix-next, iy-next)  
critical (x axis)
```

Appl : Lib. Hyper , Compiler Directives , Prog. Lang.
(Easy, port) (Easy, portable) (Port, Flexi)
OpenACC

```
#pragma acc parallel / loop gang / loop vector
```

Maintain existing code , seq $\rightarrow$ annotated parallel
Compiler converts based on CPU arch (as Hint: parallelize)

```
$ pgcc-fast -Minfo=accel -ta=tesla main.c
```

Data Clauses

```
#pragma acc exit data copyout (c[0:N]) delete (a,b)
```

Mem. Mgmt

Gang - Worker - Vectors :: block - warp - thread // auto-calc values
Analyze $\rightarrow$ Parallelize $\rightarrow$ Optimize

Check : openacc.org / Prog Guide

Hybrid Parallelization

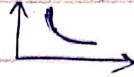
Multi Computers:

FrontEnd $\rightarrow$ Switched Eth $\rightarrow$ { GPUs } $\leftarrow$ Myrinet switch

Distributed Computing

Performance = Spanning job across multicomp.

MPI vs MPI + Open MP



- Thread safety, Cache updates

Job $\rightarrow$ Chunks of MPI $\rightarrow$ parallelized using OpenMP

Halo Zones

MPI-THREAD - SINGLE = seq., master th.

FUNNELED = one th. calls MPI lib

SERIALIZED = all th. make lib calls

dev. control so 1 thread at a time

MULTIPLE = all th. make any lib call

⊗ Synchronization

GPU acceleration and parallelization

GPU $\rightarrow$ CPU (cuda)

CPU $\rightarrow$ remote CPU (mpi)

remote CPU $\leftarrow$ mpi

remote CPU $\rightarrow$ GPU (cuda)

Nvidia GPUDirectTM (1 cpu n gpu) cuda $\geq$ 5.1, RDMA
gpu $\rightarrow$ gpu via pcie switch

ExaScale Computing = 10^{18} FLOPS

Folding @ Home : simulating protein dynamics